

AI Operations (AIOps) — Module 1 Assignment

Dhanush DA24B019

August 2026

1 Question 1 — Technical Debt Diagnosis

1.1 Mapping each scenario to a hidden-technical-debt category

- (a) **Changing the delivery-time rounding logic hurt the unrelated “favorite restaurants” feature.**

This is **Entanglement (CACE)**. A model’s features are not kept in isolated compartments; they are fused into one shared set of weights, so altering one can silently shift predictions tied to a feature with no logical connection to it.

- (b) **A marketing dashboard team silently reads the model’s raw output table.**

This is **Undeclared Consumers**. A team began depending on an internal artifact without informing its owners, so the ML team can no longer safely change that output without risking an invisible breakage elsewhere.

- (c) **The training pipeline is 14 undocumented shell scripts with no orchestration tool.**

This is **Configuration and Glue-Code Debt**. With no declared entry point or execution order, the pipeline only works because someone remembers how it is wired — a classic “pipeline jungle.”

1.2 Proposed mitigation

For scenario (c), consolidate the 14 scripts into a single **MLflow Project** (MLproject file + pinned environment.yml), so the pipeline runs as one declared, reproducible command instead of undocumented tribal knowledge:

```
mlflow run .
```

2 Question 2 — MLflow Experiment Comparison

2.1 Run comparison table

Figure 1: MLflow UI run-comparison table for all six runs.

Run	LR	Batch size	Final train acc.	Final val. acc.	Final train loss
1	0.001	32	1.000	0.943	0.002
2	0.001	128	1.000	0.935	0.014
3	0.01	32	0.978	0.924	0.366
4	0.01	128	0.991	0.931	0.094
5	0.1	32	0.978	0.926	3.225
6	0.1	128	0.985	0.931	0.800

Table 1: MLflow 6-run comparison for the MNIST MLP sweep.

2.2 Analysis

The best-performing run is **lr = 0.001**, **batch_size = 32**, reaching a validation accuracy of 0.943 — the highest of all six runs. Its learning rate is small enough for stable convergence over 15 epochs without the instability seen at higher rates.

There is clear evidence of overfitting at this same setting: final training accuracy reaches 1.000 (a perfect fit) while validation accuracy plateaus at 0.943, a training-validation gap of roughly 5.7 points. The model has enough capacity to memorize the training set, but that memorization stops translating into generalization past a certain point.

At the opposite extreme, **lr=0.1** shows a different failure mode entirely: both training and validation performance are worse (train loss as high as 3.225), indicating the learning rate is too large for the optimizer to converge properly within 15 epochs — this is underfitting from optimization instability, not overfitting.

Comparing the two hyperparameters, **learning rate has the larger effect on performance**. Training loss swings from 0.002 to 3.225 across the three learning rates (a $\sim 1600\times$ range) while final validation accuracy only moves from 0.924 to 0.943 across all six runs (a ~ 2 -point range). Batch size, by contrast, produces comparatively minor differences at a fixed learning rate. This suggests the optimizer’s step size is the dominant factor governing both convergence quality and the degree of overfitting in this setup.

2.3 Logging code

```
mlflow.log_param("model_type", "MLPClassifier")
mlflow.log_param("dataset", "MNIST")
mlflow.log_param("learning_rate_init", lr)
mlflow.log_param("batch_size", bs)
mlflow.log_param("hidden_layer_sizes", "(100,)")
mlflow.log_param("n_epochs", N_EPOCHS)

mlflow.log_metric("train_loss", train_loss, step=epoch)
mlflow.log_metric("val_accuracy", val_accuracy, step=epoch)

mlflow.log_metric("final_train_accuracy", final_train_acc)
mlflow.log_metric("final_val_accuracy", val_accuracy)
```